AI Engineering and Claude Architect · Part V, Certification

Certification, Lesson 2

Reading the Agent SDK and the CLI

The course builds in Java, but the Agent SDK ships for Python and TypeScript, and the exam names its options directly. You don't need to write this code from memory. You do need to recognize it, and to know every flag at a glance.

Module	AI13 of 13
Lesson	2 of 5
Prerequisites	AI06 L2, L6, L7, L8 and AI12 L5
Exam relevance	**Core: TS 1.3, 1.5, 1.7, 3.6.** Also scenarios S1, S3, S4
Reading time	~30 minutes

Contents

1. Reading the Agent SDK and the CLI

Lesson 2 of 5 · Certification

Reading the Agent SDK and the CLI

Key idea

The Agent SDK gives you the agent loop, tools and sessions as options on `query()`. A coordinator needs the subagent tool in its allowed tools, each subagent is an `AgentDefinition`, hooks are callbacks that run in your code, and sessions continue, resume or fork. The CLI exposes the same ideas as flags.

> *Before: agent loops, multi-agent patterns, sessions and hooks, taught in AI06. Teaches: how those look in the SDK's Python and TypeScript, and a full flag sheet. Used by: L4, L5 and the mocks.*

The hook: a syntax item in a Java developer's exam

You learned this course's agent loop in Java, where you own `while (stop_reason == tool_use)`. Then a scenario says: "the coordinator has `allowed_tools=["Read", "Grep"]`, and its subagents never run." You've got the concept, but the code shape is unfamiliar, and scenarios S1, S3 and S4 all say "using the Claude Agent SDK". This lesson closes that gap, at reading level.

Intuition: hiring a contractor or building a team

The Messages API is hiring one worker and managing each step yourself: you run the loop and the tools. The Agent SDK is hiring a contractor that already has a crew and tools, and giving them the rules: which tools are approved, which specialists exist, and which checks are mandatory. You set the options, and the loop runs inside.

Mechanism: one options object

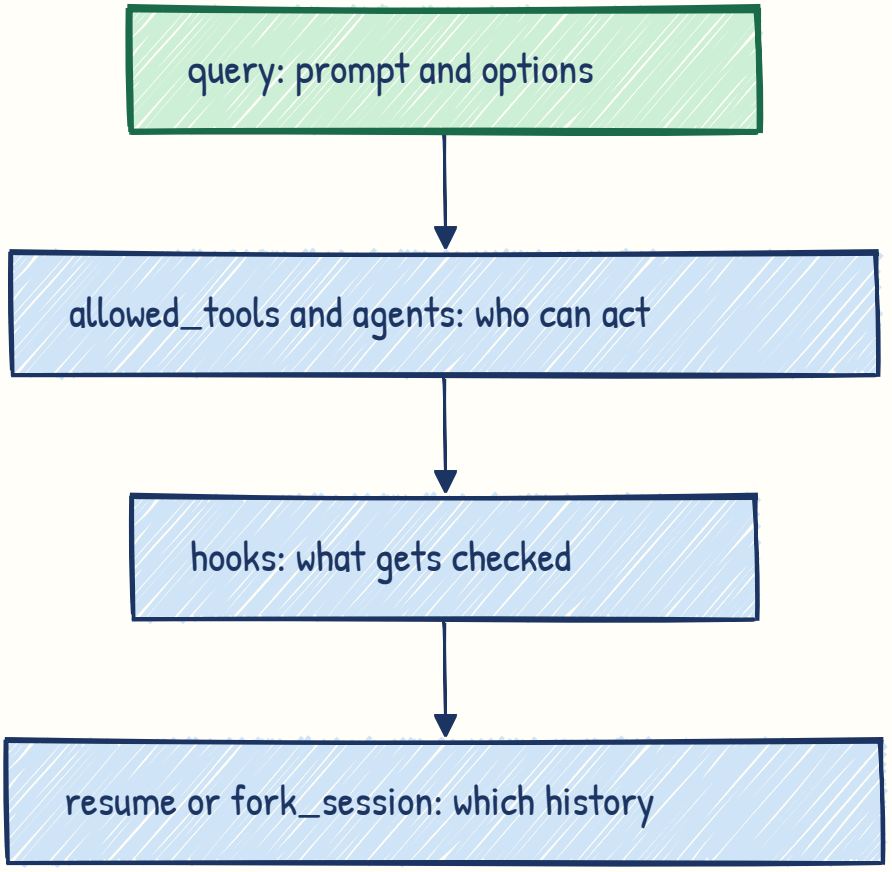


Figure 1. The SDK's main parts, as options on one call. Your code supplies configuration and callbacks, and the SDK runs the loop.

Subagents, as the guide teaches them and as they are now

```
from claude_agent_sdk import query, ClaudeAgentOptions, AgentDefinition

options = ClaudeAgentOptions(
    allowed_tools=["Read", "Grep", "Agent"],          # guide's name: "Task"
    agents={
        "code-reviewer": AgentDefinition(
            description="Reviews code for security. Use for security reviews.",
            prompt="You are a security reviewer. Report each finding with its file.",
            tools=["Read", "Grep"],                   # this subagent's own tool set
        )
    },
)
async for message in query(prompt="Review the auth module", options=options):
    ...
```

Part	What it decides	Exam point
`allowed_tools` with the spawning tool	Whether the coordinator can delegate without a permission prompt	TS 1.3
`description`	**When** the subagent is chosen	It works like a tool description, so it's the first thing to fix when delegation misfires
`prompt`	The subagent's system prompt	Its role and behavior
`tools`	What it can do. If omitted, it gets all available tools	TS 2.3: restrict to the role
The spawning tool's prompt	**The only channel from parent to subagent**	TS 1.3: pass findings there

Task or Agent? **The exam's answer: `allowedTools` must include `"Task"`.** Build today: the docs use `"Agent"` in `allowed_tools` and `tool_use` blocks, while the `system:init` tools list still shows `"Task"`. Before Claude Code v2.1.63, `tool_use` blocks also said `"Task"`. Same mechanism, different name.

What a subagent receives. The documented table is worth memorizing:

Receives	Doesn't receive
Its own system prompt and the spawning tool's prompt	The parent's conversation history or tool results
Project `CLAUDE.md`, when settings load it	The parent's system prompt
Tool definitions, inherited or the subset in `tools`	Skill content, unless listed in `skills`

Build today: a **fork** subagent is the exception. It copies the conversation so far, and `subagent_type: "fork"` starts one. The exam's rule stays that subagents don't inherit context, and the guide's "fork" refers to `fork_session`, below.

Parallel subagents. Emit several spawning tool calls in a single coordinator response (TS 1.3). Each one's intermediate work stays inside it, and only its final message returns.

Hooks

Hooks are plain async callbacks. This course's lab ran these two directly, with no model, against SDK version 0.2.178:

```
async def refund_gate(input_data, tool_use_id, context):
    amount = input_data["tool_input"].get("amount", 0)
    if amount > 500:
        return {"hookSpecificOutput": {
            "hookEventName": input_data["hook_event_name"],
            "permissionDecision": "deny",
            "permissionDecisionReason": "Over $500: call escalate_to_human.",
        }}
    return {}          # no objection

options = ClaudeAgentOptions(hooks={
    "PreToolUse": [HookMatcher(matcher="mcp__orders__process_refund",
                               hooks=[refund_gate], on_failure="block")],
    "PostToolUse": [HookMatcher(matcher="mcp__orders__lookup_order",
                                hooks=[normalize_order])],
})
```

Real output from `code/ai13-exam/sdk_lab.py`:

```
refund 120: no objection
refund 800: deny
normalized: {'order_id': 'A1', 'placed_at': '2026-01-01T00:00:00Z', 'status': 'shipped'}
normalized: {'order_id': 'B2', 'placed_at': '2026-01-01T00:00:00Z', 'status': 'delivered'}
```

The two inputs used a Unix timestamp and an ISO 8601 time with an offset, and numeric status codes, which is the heterogeneous data of TS 1.5. The model would see one format.

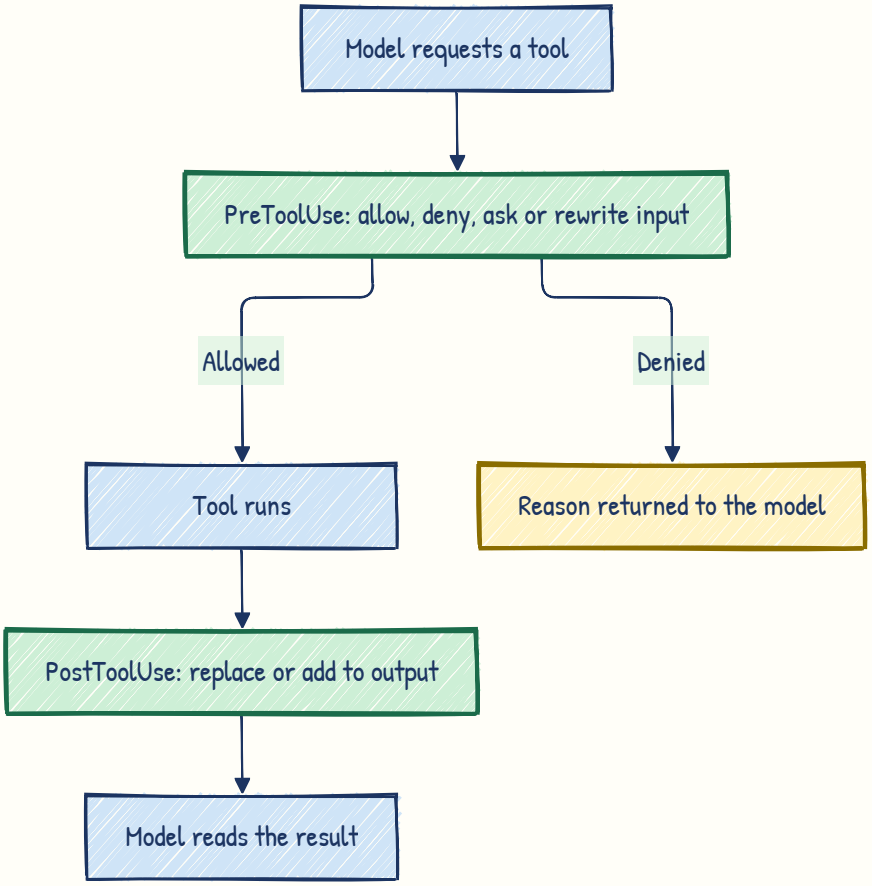


Figure 2. Where each hook sits in a tool call. PreToolUse can stop the call, and PostToolUse changes what the model reads.

Fact	Documented detail
Deny	`permissionDecision: "deny"`, and the reason goes to the model
Rewrite input	`updatedInput` inside `hookSpecificOutput`
Replace output	`updatedToolOutput` on PostToolUse, for any tool. The older `updatedMCPToolOutput` covers MCP tools only
Add context	`additionalContext`, appended to the tool result
Several hooks disagree	`deny` beats `defer`, which beats `ask`, which beats `allow`
Matcher	Matches the tool name, not arguments. MCP tools are `mcp__<server>__<action>`
A hook crashes	The action goes ahead on most events, unless you set `on_failure="block"`. Set it on a guard

The hidden catch. A `Write|Edit` matcher doesn't fire if Claude writes the same file through Bash, as the docs warn. That's why a guard hook isn't the only layer you should have, as AI09 explains.

Sessions

Goal	Python	TypeScript	CLI
Most recent in this directory	`continue_conversation=True`	`continue: true`	`-c`, `--continue`
A specific session	`resume=session_id`	`resume: sessionId`	`-r`, `--resume <id or name>`
A branch that keeps the original	`resume=id, fork_session=True`	`resume, forkSession: true`	`--resume <id> --fork-session`
Multi-turn in one process	`ClaudeSDKClient`	`continue: true` on each `query()`	Interactive session

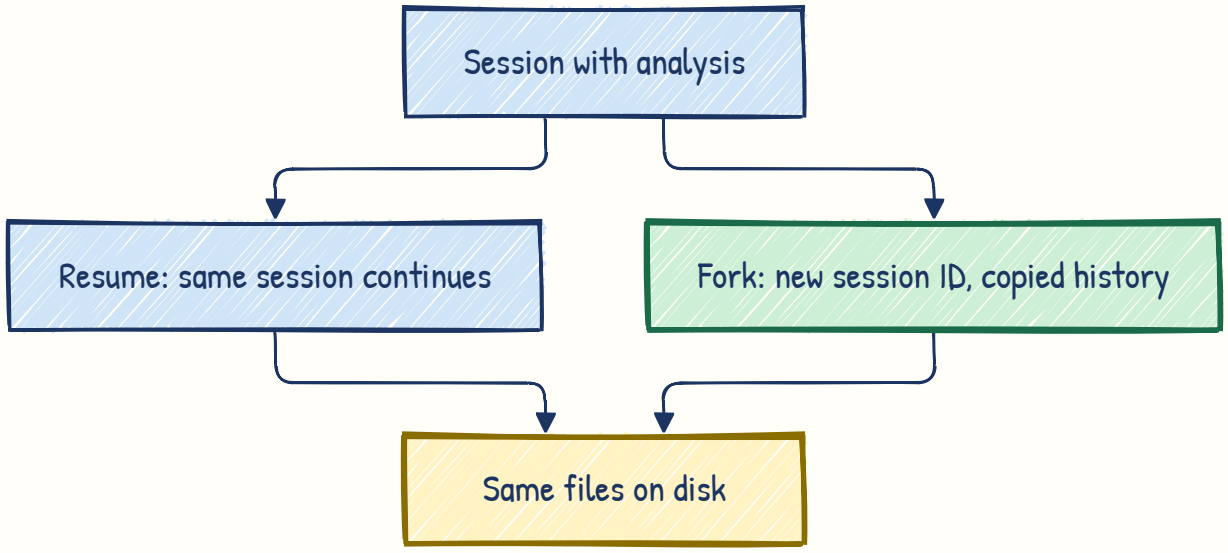


Figure 3. Resume adds to one history. A fork copies it into a new session, while the original stays as it was and the files on disk stay shared.

→ **Forking branches the conversation, not the filesystem.** File edits are real, and every session in that directory sees them.

→ The session ID comes from the result message's `session_id`.

→ Sessions are files on one machine. For another host, the docs' simplest option is to capture results as application state and pass them into a new session's prompt, which matches the guide's "fresh start with a structured summary".

→ Build today: `--continue` skips sessions made with `claude -p` or the SDK, so scripted runs usually pass an ID.

Spend and growth limits

These aren't exam objectives, but one might appear as a distractor or a production detail. `max_turns` stops a run (`error_max_turns`), `max_budget_usd` caps estimated spend, and `CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH` and `CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS` cap the tree of agents. In TS 1.1 terms, a turn limit is a safety net, not the normal stop.

Why the obvious readings fall short

Tempting reading	What's wrong	Correct reading
"`allowed_tools` is the agent's tool list"	It auto-approves tools	Restrict a subagent with `tools`. The CLI's `--tools` restricts built-ins
"A subagent can see the parent's tool results"	Not unless it's a fork	Pass them in the prompt
"PostToolUse can block the call"	The tool has already run	Block in PreToolUse
"Forking gives me a separate checkout"	It copies only the history	Use worktrees for files
"`AgentDefinition.prompt` is the task"	It's the system prompt	The task arrives via the spawning tool's prompt

The CLI flag sheet

Candidates report CLI flags as a hard area. **Bold rows are in the guide** and are the ones to memorize. The rest are in the current CLI reference and help you recognize distractors.

Flag	Meaning	Note
`-p`, `--print`	Non-interactive: answer and exit	Prevents CI hangs
`--output-format json`	Structured result envelope. Others: `text`, `stream-json`	Print mode
`--json-schema`	Validated output matching a schema, after the work is done	Print mode
`--resume`, `-r`	Resume by ID or name	With no value, a picker
`--fork-session`	New session ID when resuming	With `--resume` or `--continue`
`--continue`, `-c`	Most recent in this directory	Skips `-p` and SDK sessions
`--name`, `-n`	Name a session for later `--resume <name>`	Supports the guide's named sessions
`--allowedTools`	Tools that run without a prompt	Not a restriction
`--disallowedTools`	Deny rules. A bare name removes the tool	Scoped form: `Bash(rm *)`
`--tools`	Restricts available built-in tools	`""` disables all
`--permission-mode`	`default`, `acceptEdits`, `plan`, `auto`, `dontAsk`, `bypassPermissions`	`plan` starts in plan mode
`--max-turns`	Limits agentic turns	Print mode
`--max-budget-usd`	Caps estimated spend	Print mode
`--mcp-config`	Loads MCP servers from JSON	
`--strict-mcp-config`	Uses only `--mcp-config` servers	Useful in CI
`--append-system-prompt`	Adds to the default prompt	Keeps Claude Code's defaults
`--system-prompt`	Replaces the whole prompt	Removes the defaults
`--bare`	Skips hooks, skills, MCP, `CLAUDE.md` discovery	Then project context isn't loaded

Flag	Meaning	Note
`--agents`	Defines subagents as JSON	
`--add-dir`	Adds working directories	Most `.claude/` config isn't discovered there
`--no-session-persistence`	Doesn't save the session	Print mode
`--model`	Sets the model	Alias or full name

In-session commands in the guide

Command	Use	TS
`/memory`	See which memory files are loaded	3.1
`/compact`	Reduce context use	5.4
`/resume`	Pick up a session	1.7

Reading a CI command

```
claude -p "Review this diff against REVIEW.md. Report only new issues." \
  --output-format json \
  --json-schema "$(cat findings.schema.json)" \
  --allowedTools "Read,Grep,Glob" \
  --max-turns 15
```

Check it against the exam's reasoning: `-p` means no hang, JSON plus a schema means parseable findings for inline comments, and the project `CLAUDE.md` supplies standards. Adding `--bare` would break that last part. This command wasn't run in this course, to avoid spending your subscription. AI12 L5 quotes the flags' documented behavior.

The Agent SDK in production

This course's reference design for S1 (AI06 L9) maps directly to the SDK. **This is a recommended reference design, not a specific company's system.**

Baseline: `query()` with MCP tools and a prompt. **New problem:** refund rules hold only most of the time. **Improved:** add PreToolUse gates and PostToolUse normalization. **New problem:** long sessions and verbose lookups. **Production:** also add case facts, trimmed tool results and structured handoff.

Component	Why it exists	How it connects
Coordinator `query()`	Runs the loop	Calls MCP tools and the spawning tool
`AgentDefinition` per specialist	Focused tools and prompts	Called by description
MCP servers	Backend access	`mcp_servers`, or `.mcp.json`
Hooks	Guarantees and normalization	Callbacks around each tool call
Session store or state	Resume after failure	Session ID or your own state

Failure	Result	Mitigation
Guard hook crashes	Action proceeds on most events	`on_failure="block"`, and let errors propagate
Hook times out	PreToolUse blocks the call	Set `timeout`, keep guards fast
Subagent lacks a finding	Wrong or partial work	Pass findings in the prompt
Delegation never happens	Coordinator does it all	Fix `description`, or name the agent in the prompt

Java: what stays the same

There's no official Java Agent SDK in this course's sources, so AI08 built the same loop on `anthropic-java`. The concepts transfer directly: `allowed_tools` is your tool registry, hooks are the checks around tool dispatch, and sessions are your stored message list. The exam doesn't require Python or TypeScript fluency, as far as the guide says, but its appendix names these SDK concepts. This lesson's code was run with the Python SDK and uses no Java, deliberately.

Trade-offs

The SDK saves building the loop, tools and sessions, but it bundles Claude Code's behavior, and you take on its version changes. A direct API loop, as in AI08, gives full control and more code to own. **Don't use an agent at all** for a fixed pipeline, as AI06 L1 teaches.

> *Common mistakes*
>
> Don't memorize flag spellings you've never reasoned about. Learn each flag as a decision: what does it stop from happening? `-p` stops the hang, `--json-schema` stops unparseable output, and `--fork-session` stops the original from changing.

> *Tip*
>
> For a Task-versus-Agent item, look for the option written in the guide's terms. If only one option mentions the spawning tool in `allowedTools`, the rest of the sentence is what's being tested.

Certification lens

> *Key idea*
>
> Supports **TS 1.3, 1.5, 1.7, 2.3, 3.6**, and the guide appendix's SDK and CLI lists. Reading level is my judgment of what's needed. The guide doesn't state whether items show code. SDK options and hook outputs were checked against current docs and an installed SDK. No model call was made.

These are original practice questions, not real or leaked.

Q Q1 · Exam style, scenario: Multi-Agent Research System · Easy. The coordinator's `allowedTools` is `["WebSearch", "Read"]`, and it never spawns its defined subagents. What's missing?

A. A `subagent_type` field on each definition

B. `"Task"` in `allowedTools`

C. A shared memory store

D. A larger `max_turns`

A **Answer: B.** The guide requires it. *Build today:* the current name is `Agent`.

Q Q2 · Exam style, scenario: Customer Support Resolution Agent · Medium. `lookup_order` and `get_customer` return dates in different formats, and the agent sometimes compares them wrongly. What's the most reliable fix?

A. Ask the model in the system prompt to normalize dates

B. Add a date-normalizing subagent

C. A PostToolUse hook that normalizes the results before the model reads them

D. A PreToolUse hook that rewrites the tool's arguments

A **Answer: C.** It's deterministic and happens before the model sees the data. PreToolUse sits before the tool returns anything.

Q Q3 · Exam style, scenario: Developer Productivity with Claude · Medium. You've analyzed a codebase, and you want to compare two refactoring strategies without losing that analysis. Which is best?

A. Resume twice in the same session

B. `fork_session` from the analysis, once per strategy

C. Start two new sessions and redo the analysis

D. Spawn subagents that inherit the analysis automatically

A **Answer: B.** Each fork gets the shared baseline, and the original stays unchanged. Remember that forks still share the filesystem.

Q Q4 · Exam style, scenario: Claude Code for Continuous Integration · Medium. Which command suits a pipeline that posts findings as inline PR comments?

A. `claude "review this"`

B. `claude -p "review" --output-format json --json-schema "$S"`

C. `claude -c "review"`

D. `claude --resume review --bare`

A **Answer: B.** It's non-interactive and produces parseable, schema-checked output. A would wait for input, and C needs a prior session and has no schema.

Q Q5 · Exam style, scenario: Multi-Agent Research System · Hard. Choose the **two** statements that are true of a non-fork subagent.

A. It can read the parent's earlier tool results

B. The spawning tool's prompt is the only channel for passing it context

C. Its intermediate tool calls stay in its own context, and its final message returns

D. It shares its memory with other subagents

E. The coordinator's system prompt is included

A **Answer: B and C.** Isolation is the point. The documented table excludes A and E, and D is the assumed-inheritance trap.

Revision sheet

→ SDK: `query()` plus options. Allowed tools auto-approve, and `tools` restrict a subagent.

→ `AgentDefinition`: description decides when, prompt is the system prompt, tools restrict.

→ Exam says `Task`, current docs say `Agent`.

→ Subagents get only their prompt, and no parent history. Pass findings explicitly.

→ Parallel: several spawning calls in one response.

→ PreToolUse: `permissionDecision` deny, or `updatedInput`.

→ PostToolUse: `updatedToolOutput` replaces what the model reads.

→ Failed guard hooks let actions proceed unless `on_failure="block"`.

→ `continue` is most recent, `resume` is specific, `fork_session` branches history and not files.

→ CI: `-p`, `--output-format json`, `--json-schema`, and project `CLAUDE.md`.

Your notes